

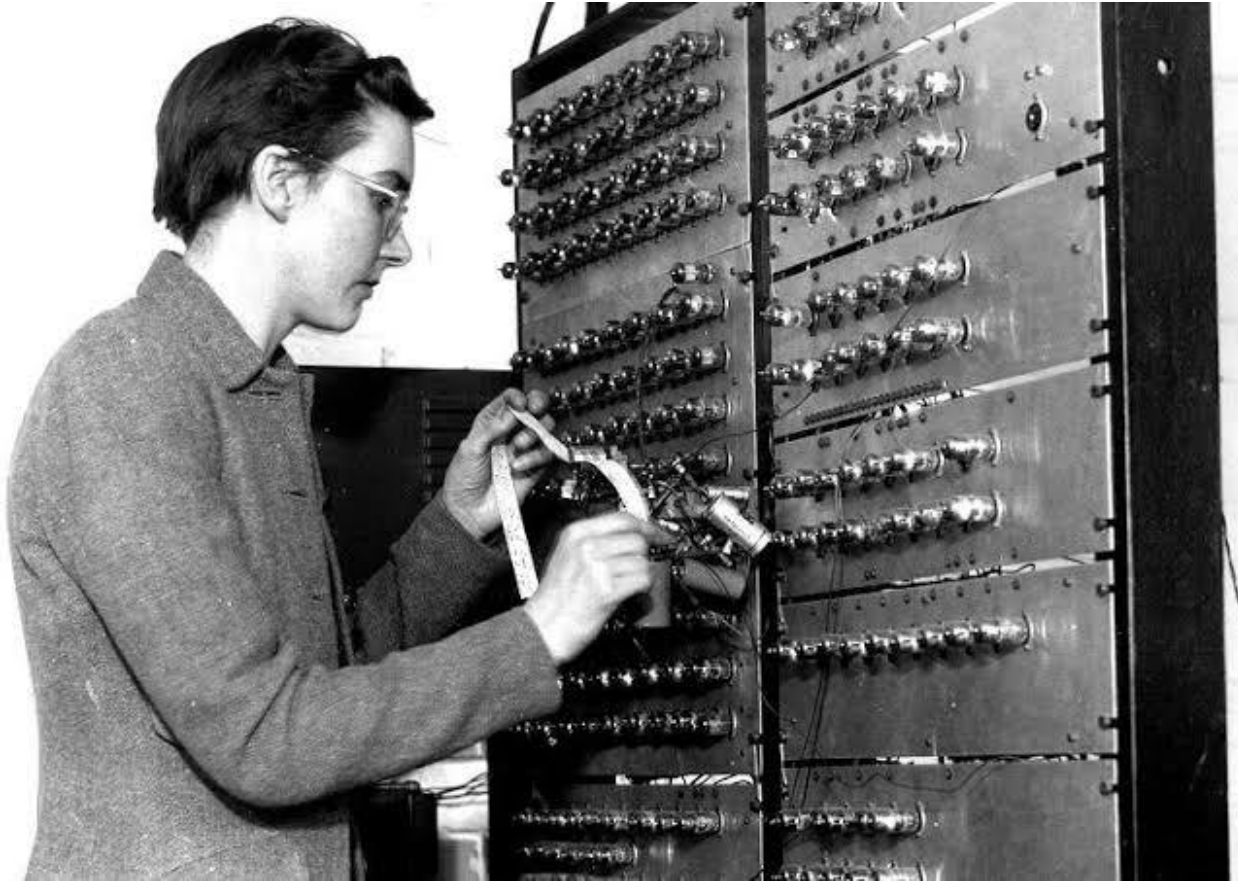
CSCI 210: Computer Architecture

Lecture 9: Logical Operations

Stephen Checkoway

Slides from Cynthia Taylor

CS History: Kathleen Britton



- Applied mathematician and computer scientist
- Wrote the first assembly language and assembler in 1947
- Collaborated with Andrew Booth to develop three early computers: the ARC (Automatic Relay Calculator), SEC (Simple Electronic Computer), and APE(X)C
- Later worked with neural nets

Logical Operations

- Instructions for bitwise manipulation

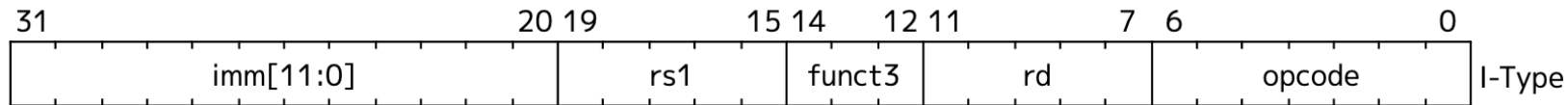
Operation	C	Java	RISC-V
Shift left	<<	<<	slli
Shift right	>>	>>>	srlr
Bitwise AND	&	&	and, andi
Bitwise OR			or, ori
Bitwise XOR	^	^	xor, xori
Bitwise NOT	~	~	xori

- Useful for extracting and inserting groups of bits in a word

Shift left logical

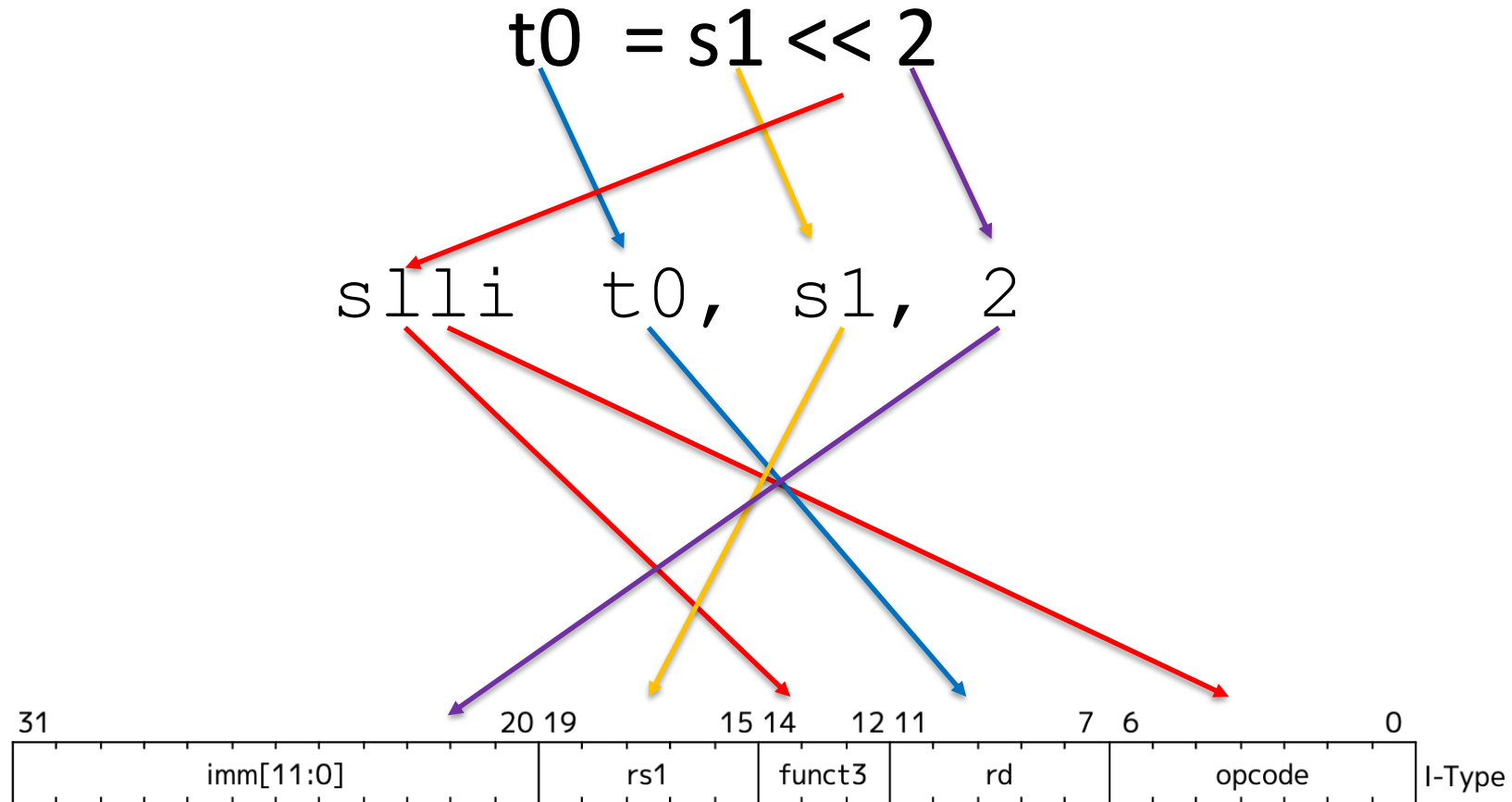
- Shifts the integer left and fills the right-most bits with 0
- $0110\ 1001 \ll 2$ (in 8 bits)
 - Most significant 2 bits are **dropped**
 - 2 0s are **added** to become the new least significant bits
 - Result: ~~01~~ 1010 01**00** => 1010 0100
- Shifting longer integers (e.g., doublewords) works the same
 - 0011011011101001000001011010101000100101101101100101100101101100 << 37
gives
1011011011001011001011011000000000000000000000000000000000000000

Logical Shift Operations (I-type)



- 64-bit integers can only be shifted by up to 63 bits
- `imm[5:0]`: how many bits to shift `rs1` left or right
- Shift left logical immediate (`slli`)
 - Shift `rs1` left by `imm[5:0]` bits and fill the right-most bits with 0s
 - `slli dest, src, n` multiplies `src` by 2^n
- Shift right logical immediate (`srl`)
 - Shift `rs1` right by `imm[5:0]` bits and fill with left-most bits with 0s
 - `srl dest, src, n` divides `src` by 2^n (unsigned only)

RISC-V shift instructions



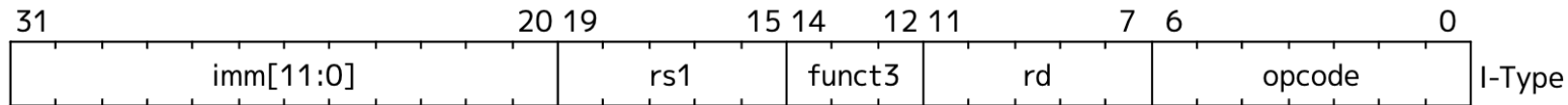
Shift right logical

- $1010\ 1001 \ggg 3$ (in 8 bits)
 - Least significant 3 bits are **dropped**
 - 3 0s are **added** to become the most significant bits
 - Result: $0001\ 0101$ ~~001~~ $\Rightarrow 0001\ 0101$

Shift right arithmetic

- Shift right and copy the sign bit (rather than 0) into the new positions
- 1010 1001 >> 3 in 8 bits
 - Least significant 3 bits are **dropped**
 - 3 1s are **added** because the MSB is 1 to become the most significant bits
 - Result: **1111** 0101 ~~001~~ => 1111 0101

Shift Operations (I-type)



- 64-bit integers can only be shifted by up to 63 bits
- `imm[5:0]`: how many bits to shift `rs1` left or right
- For `slli` and `srl`, the upper immediate bits are zero
`imm[11:6] = 000000`
- For `srai`, bit 30 of the instruction is 1; i.e.,
`imm[11:6] = 010000`

A new op HEXSHIFTRIGHT shifts hex numbers right by a digit. HEXSHIFTRIGHT i times is equivalent to

- A. Dividing by i
- B. Dividing by 2^i
- C. Dividing by 16^i
- D. Multiplying by 16^i

Remember Boolean Operations?

- AND, OR, NOT . . .
- Now we'll apply them to bits!
- Just think of 1 as True, and 0 as False

And Truth Table

	<i>0</i>	<i>1</i>
<i>0</i>	0	0
<i>1</i>	0	1

AND Operations

- Useful to mask bits in a word
 - Select some bits, clear others to 0

and t_0 , t_1 , t_2

t_2 ...0000 0000 0000 0000 0000 1101 1100 0000

t_1 ...0000 0000 0000 0000 0011 1100 0000 0000

t_0 ...0000 0000 0000 0000 0000 1100 0000 0000

The upper 32 bits of t_0 , t_1 , and t_2 are not shown here; they're all 0

AND identities (for a single bit)

- $x \& 0 =$
- $x \& 1 =$

$$\begin{array}{r} 0110 \quad 1001 \\ \& \quad 1100 \quad 0111 \end{array}$$

$$A. 0001 \quad 0000$$

$$B. 0100 \quad 0001$$

$$C. 1010 \quad 1110$$

$$D. 1110 \quad 1111$$

If we want to zero out bits* 3 – 0 in a byte we should AND with

A. 0000 0000

B. 0000 1111

C. 1111 0000

D. 1111 1111

*MSB (bit 7) is on the left,
LSB (bit 0) is on the right

One way to represent colors is to specify the amount of **red**, **green**, and **blue** (RGB) that makes up the color. The three color “channels” are often packed into a 32-bit integer as follows:

00000000 **rrrrrrrr** **gggggggg** **bbbbbbbb**

where each color is represented in 8 bits. If `color` is a 32-bit RGB color, which high-level expression extracts the green channel as an 8-bit value in the range 0–255?

A. `green = (color >> 8) & 0xFF`

B. `green = (color >> 16) & 0xFF`

C. `green = (color & 0xFF) >> 8`

D. `green = (color & 0xFF00) >> 16`

E. More than one of the above (which?)

Assume `color` is stored in `t0` and `green` should be extracted to `t1`. Which sequence of RISC-V instructions corresponds to $\text{green} = (\text{color} \gg 8) \ \& \ 0\text{xFF}$

A. `slli t1, t0, 8`
 `andi t1, t1, 0xFF`

B. `srlr t1, t0, 8`
 `andi t1, t1, 0xFF`

C. `andi t1, t0, 0xFF`
 `slli t1, t1, 8`

D. `andi t1, t0, 0xFF`
 `srlr t1, t1, 8`

Or Truth Table

	<i>0</i>	<i>1</i>
<i>0</i>	0	1
<i>1</i>	1	1

OR Operations

- Useful to set bits in a word
 - Set some bits to 1, leave others unchanged

or t0, t1, t2

t2 ...0000 0000 0000 0000 0000 1101 1100 0000

t1 ...0000 0000 0000 0000 0011 1100 0000 0000

t0 ...0000 0000 0000 0000 0011 1101 1100 0000

The upper 32 bits of t0, t1, and t2 are not shown here; they're all 0

OR Identities (for a single bit)

- $x \mid 0 =$

- $x \mid 1 =$

0110 1001
| 1100 0111

A. 0001 0000

B. 0100 0001

C. 1010 1110

D. 1110 1111

Recall RGB:

00000000 rrrrrrrr gggggggg bbbbbbbb

If r , g , and b are values in the range 0–255, how can we construct the RGB value, c , whose channels are r , g , and b ?

A. $c = (r \ll 16) \mid (g \ll 8) \mid b$

B. $c = (r \ll 24) \mid (g \ll 16) \mid (b \ll 8)$

C. $c = (r \gg 8) \mid (g \gg 16) \mid (b \gg 24)$

D. $c = (r \gg 16) \mid (g \gg 24) \mid (b \gg 32)$

E. More than one of the above (which?)

XOR Truth Table

	<i>0</i>	<i>1</i>
<i>0</i>	0	1
<i>1</i>	1	0

XOR Operations

- Exclusive OR (written $x \oplus y$ or $x \wedge y$)
 - Set output bits to one only if the corresponding input bits are not the same

xor t0, t1, t2

t2 ...0000 0000 0000 0000 0000 1101 1100 0000

t1 ...0000 0000 0000 0000 0011 1100 0000 0000

t0 ...0000 0000 0000 0000 0011 0001 1100 0000

The upper 32 bits of t0, t1, and t2 are not shown here; they're all 0

$$\begin{array}{r} 0110 \quad 1001 \\ \text{XOR } 1100 \quad 0111 \end{array}$$

A. $0001 \quad 0000$

B. $0100 \quad 0001$

C. $1010 \quad 1110$

D. $1110 \quad 1111$

XOR Identities (for a single bit)

- $x \text{ XOR } 0 =$
- $x \text{ XOR } 1 =$

NOT operations

- Inverts all the bits in a word
 - Change 0 to 1, and 1 to 0

RISC-V does not need a `not` instruction because we
can use _____ for `not t1, t2`

A. `xori t1, t2, 0`

B. `xori t1, t2, 1`

C. `xori t1, t2, -1`

D. It does require a separate `not` instruction

E. None of the above are correct

10 & 7

- A. 0
- B. 2
- C. 7
- D. 10
- E. None of the above

Set bit 4 in byte x to 1, leaving the rest of the bits unchanged

$$A.x = x \text{ AND } 0001_0000$$

$$B.x = x \text{ AND } 1110_1111$$

$$C.x = x \text{ OR } 0001_0000$$

$$D.x = x \text{ XOR } 111_01111$$

Invert bits 2–0 of x

$$A.x = x \text{ AND } 0000 \ 0111$$

$$B.x = x \text{ OR } 0000 \ 0111$$

$$C.x = \text{NOT } x$$

$$D.x = x \text{ XOR } 0000 \ 0111$$

Find the ones' complement of x (in 8 bits)

A. $x \text{ XOR } 0000 \ 0000$

B. $x \text{ XOR } 1111 \ 1111$

C. $x \text{ XOR } 1111 \ 1110$

D. $x \text{ OR } 1111 \ 1111$

RISC-V I-type instructions (like `andi`, `ori`, and `xori`) have immediate values that are 12 bits.

How do you perform the logical operations AND, OR, and XOR on 64-bit doublewords?

- A. RISC-V has register versions of the instructions (`and`, `or`, and `xor`) that operate on doublewords
- B. The operations are performed 12-bits at a time by shifting the operands and results to build up the final value piece by piece
- C. Real programs never need to do that so it doesn't come up

Reading

- Next lecture: Branching instructions
 - Reading Section 2.7
- Problem Set 2 due Friday
- Lab 2 available